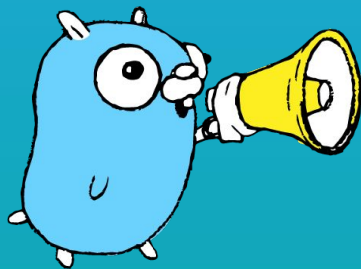




v2023.2

Создаем проект





Разделы

Структура проекта

01

Библиотеки

02

“

Disclaimer



Рекомендации носят общий характер и не учитывают специфику вашего проекта

Как это обычно происходит?



1. Прийти на проект без опыта работы с Go
2. Взять популярные на гитхабе библиотеки (gin, gorm, validator и тд)
3. Писать так, как писали до этого на php

Profit - проект стал legacy еще до релиза

Что нужно чтобы этого не случилось?



1. Выбрать основные библиотеки не по звездочкам на гитхабе
2. Обозначить в приложении слои абстракции, но не переусложнить
3. Написать с десяток небольших программ на Go, прежде чем начать писать большую

Библиотеки





- 1. Конфигурация
- 2. Контроль запуска
- 3. HTTP роутер
- 4. Логгер
- 5. БД

```
type Config struct {  
    Host      string    `config:"host"`  
    Port      uint32   `config:"port"`  
    Timeout   time.Duration `config:"timeout"`  
    Database struct {  
        URI string    `config:"database-uri,required"`  
    }  
}
```

github.com/knadh/koanf

github.com/heetch/confita

[flag](#)


```
g := &errgroup.Group{}  
g.Go(func() error { ... })  
g.Wait()
```

golang.org/x/sync/errgroup

```
func main() {  
    r := chi.NewRouter()  
    r.Use(middleware.Logger)  
    r.Get("/", func(w http.ResponseWriter, r *http.Request) {  
        w.Write([]byte("welcome"))  
    })  
    http.ListenAndServe(":3000", r)  
}
```

github.com/go-chi/chi

```
sink := zap.NewProduction()
defer sink.Sync() // flushes buffer, if any

logger := logger.Sugar()

logger.Infow("failed to fetch URL",
    // Structured context as loosely typed key-value pairs.
    "url", url,
    "attempt", 3,
    "backoff", time.Second,
)

logger.Infof("Failed to fetch URL: %s", url)
```

github.com/uber-go/zap

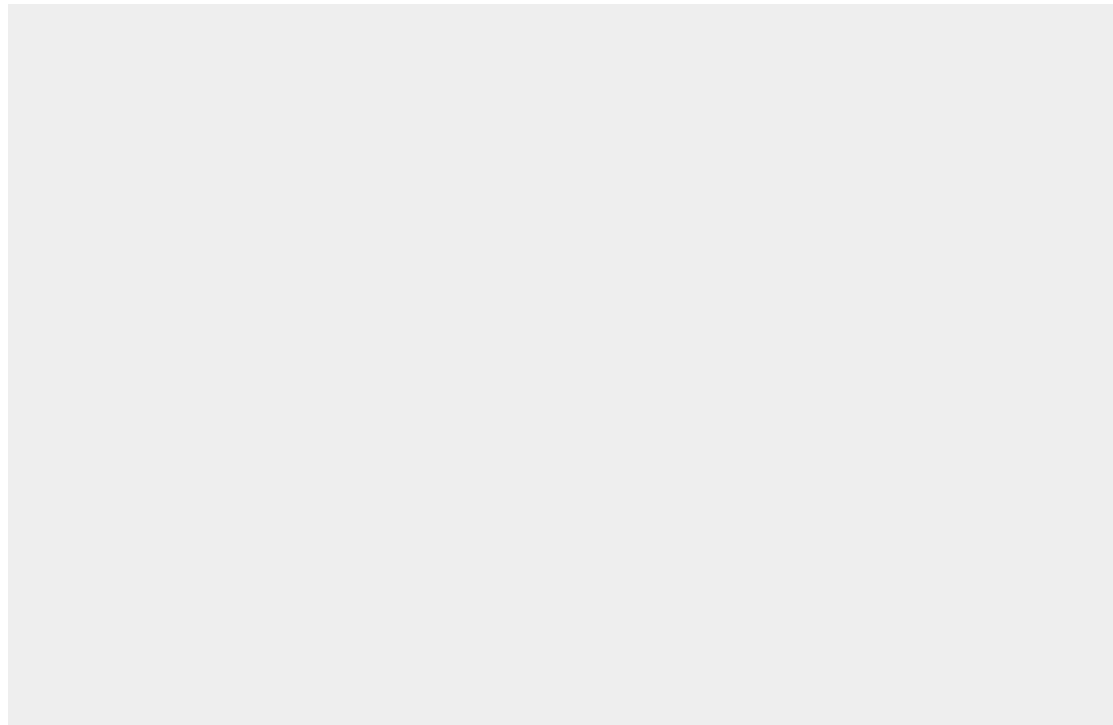
Если RDB

```
func (s *SampleStorage) SampleGetByID(ctx context.Context, id
int64) (dto.Sample, error) {
    // language=PostgreSQL
    const q = "SELECT * from sample WHERE id = $1"
    var res internal.Sample
    if err := s.db.session.Select(&res, q, id); err != nil {
        return dto.Sample{}, fmt.Errorf("could not perform
query: %w", err)
    }
    return res.ToDTO(), nil
}
```

github.com/jackc/pgx

github.com/jmoiron/sqlx

github.com/Masterminds/squirrel



github.com/golang/mock

github.com/stretchr/testify

testing

<https://threedots.tech/post/list-of-recommended-libraries/>

<https://github.com/stars/agalitsyn/lists/recommended-packages-for-go>

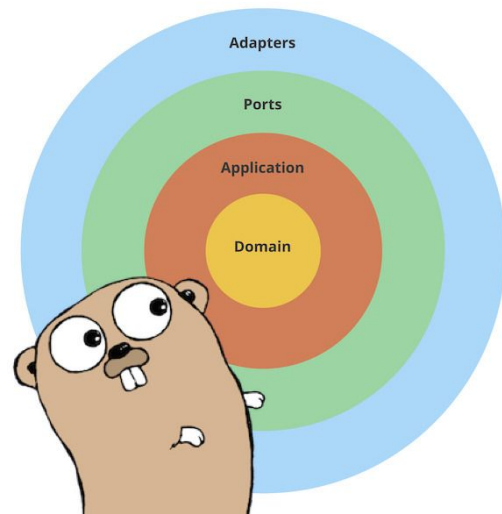
<https://github.com/stars/agalitsyn/lists/not-recommended-packages-for-go>

1

Структура проекта

Одновременно похоже на то, что есть в C#/Java/Python и тд.

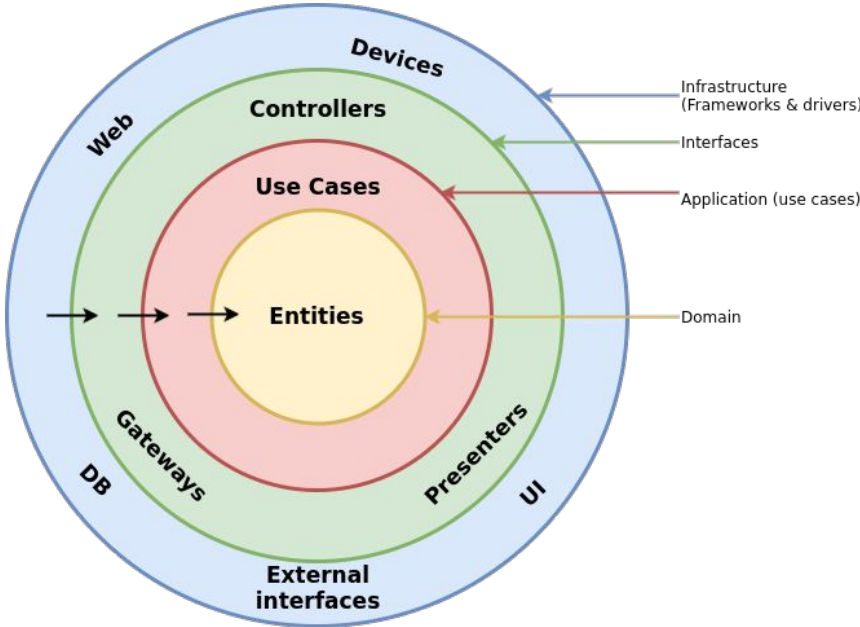
Одновременно не похоже, тк нет фреймворков



Отступление - о “чистой” архитектуре

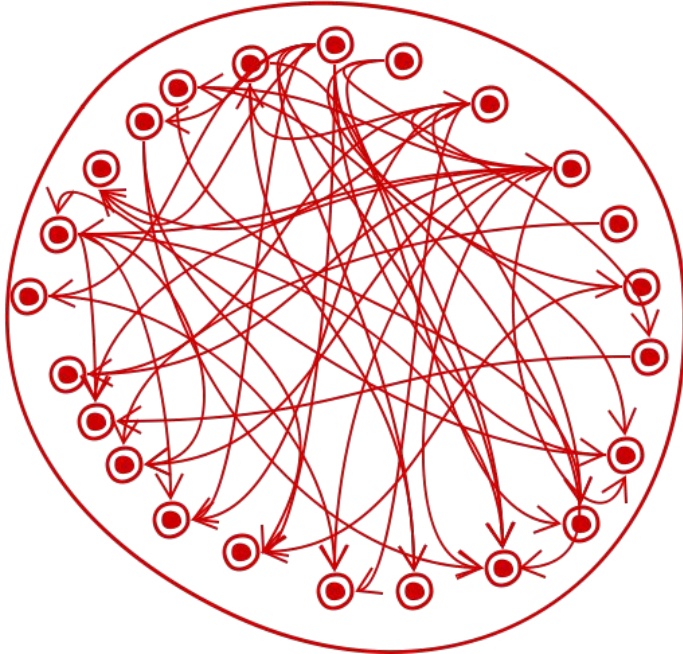


Ожидание



Based on schema from "The Clean Architecture"

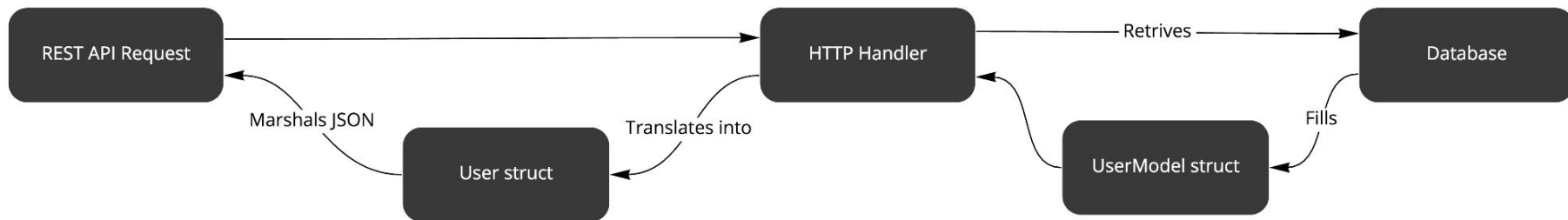
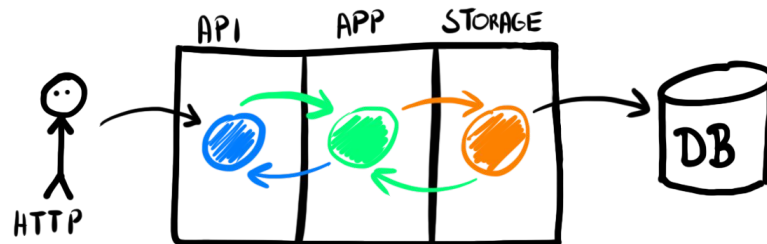
Реальность



Первоначальный уровень абстракции



1. Транспортный слой
2. Слой бизнес-логики
3. Слой хранилища



TODO

Описать файловый лейаут

Показать где что лежит, что такое модели, что такое респонс, реквест структуры, что такое структуры для базы

Прикрепить пример

Есть 3 типа структур

- Request response. Они в клиенте лежат
- DB структура которая сериализуется при вызове дб методов через scan и value методы
- Модель, она же DTO, которая не имеет структурных тегов и нужна для обмена информацией между слоями приложения

О количестве проблем со структуризацией кода, которые ждут разработчиков можно понять из доклада

<https://www.youtube.com/watch?v=IV0wrVb31Pg>

Далее можно прочитать статьи по дизайну пакетов и репозитория:

<https://rakyll.org/style-packages/>

<https://www.ardanlabs.com/blog/2017/02/package-oriented-design.html>

<https://peter.bourgon.org/go-best-practices-2016/#repository-structure>



Вопросы

=====CUT=====